

Blueprint Técnico - SaaS de Filas Digitais

Especificação completa para construção automatizada de uma plataforma multi-tenant de filas, senhas e atendimento em tempo real

Objetivo: servir como documento de arquitetura, backlog técnico e instrução de implementação do início ao fim para um agente de IA ou equipe de desenvolvimento.

Stack de referência: ASP.NET Core Web API + C# + Entity Framework Core + PostgreSQL + React/Next.js + TypeScript + SignalR + Redis + Docker.

Versão do blueprint: 1.0 | Agosto de 2026

1. Visão do produto

O produto é uma plataforma SaaS multi-tenant para gerenciamento de filas físicas e virtuais. Cada organização possui unidades, serviços, filas, atendentes e configurações isoladas. O cliente final entra na fila por QR Code, link, totem ou recepção, acompanha sua posição e recebe chamadas/notificações sem precisar permanecer fisicamente em uma fila.

1.1 Problema

- Tempo perdido em filas físicas e ambientes lotados.
- Ausência de previsibilidade de espera.
- Atendentes sem visão operacional centralizada.
- Empresas sem métricas confiáveis de demanda, espera, abandono e produtividade.
- Soluções legadas dependentes de hardware ou difíceis de adaptar a diferentes segmentos.

1.2 Proposta de valor

- Fila digital acessível por QR Code e navegador, sem instalação obrigatória.
- Atualização em tempo real da posição e das chamadas.
- Painel de TV opcional e painel de atendente.
- Multiempresa e multiunidade desde a arquitetura.
- Configuração por segmento, sem criar um software separado para cada nicho.
- Métricas de SLA, tempo médio, abandono, volume e desempenho.
- Integração futura com WhatsApp, SMS, e-mail, APIs externas e IA.

2. Escopo funcional

Módulo	MVP	Evolução
Autenticação	Login, refresh token, recuperação	SSO, MFA
Organizações	Tenant, unidade, usuários	Franquias, grupos econômicos
Serviços	Cadastro e duração estimada	Regras e prioridades avançadas
Filas	Criar, abrir, pausar, fechar	Filas combinadas e roteamento
Tickets	Emitir, chamar, concluir, cancelar	Transferência, recall, no-show
Cliente	QR/link, posição, previsão	PWA, geofence
Atendente	Próximo, iniciar, concluir	Skills e roteamento automático
Painel TV	Chamadas em tempo real	Voz, temas e mídia
Notificações	In-app	WhatsApp/SMS/e-mail
Relatórios	Resumo operacional	BI, exportação, previsão por IA
Billing	Plano manual no MVP	Stripe/Mercado Pago, cobrança recorrente
Auditoria	Eventos críticos	SIEM/retention configurável

3. Arquitetura

Adotar um monólito modular no início. Evitar microserviços prematuros. O domínio deve ser dividido em módulos claros, com dependências controladas, permitindo extração futura quando houver escala ou necessidade operacional real.

[Next.js Web Apps]

|-- Admin

```

|-- Attendant
|-- Customer Queue
|-- Public Display
    |
    HTTPS/WSS
    |
[ ASP.NET Core API ]
|-- Identity / Tenant
|-- Queue Management
|-- Ticket Lifecycle
|-- Notifications
|-- Reporting
|-- Billing
|-- SignalR Hubs
    |
+-----+-----+-----+
|         |         |         |
PostgreSQL  Redis  Background Jobs

```

3.1 Princípios

- Clean Architecture pragmática: Domain não depende de Infrastructure.
- SOLID, baixo acoplamento, alta coesão e interfaces nas fronteiras necessárias.
- CQRS leve apenas onde agrega clareza; não criar abstrações sem uso.
- Todas as operações de negócio importantes passam pela camada Application.
- Controllers finos: autenticação, validação de entrada, dispatch e resposta HTTP.
- Entidades protegem invariantes; não são simples sacos de getters/setters.
- Multi-tenancy obrigatório em todas as consultas de dados de negócio.
- Idempotência nas operações sensíveis e concorrência controlada ao chamar o próximo ticket.

4. Estrutura da solução backend

```

QueueFlow.sln
src/
  QueueFlow.Domain/
    Common/
      BaseEntity.cs
      AuditableEntity.cs
      DomainException.cs
    Entities/
      Organization.cs
      Branch.cs
      AppUser.cs
      UserBranch.cs
      Service.cs
      Queue.cs
      QueueCounter.cs
      QueueTicket.cs

```

- TicketEvent.cs
- Subscription.cs
- Notification.cs
- AuditLog.cs
- Enums/
 - QueueStatus.cs
 - TicketStatus.cs
 - TicketPriority.cs
 - UserRole.cs
 - NotificationChannel.cs
 - SubscriptionStatus.cs
- ValueObjects/
 - TenantId.cs
 - TicketNumber.cs
 - EstimatedWaitTime.cs
- Events/
 - TicketIssuedDomainEvent.cs
 - TicketCalledDomainEvent.cs
 - TicketCompletedDomainEvent.cs

- QueueFlow.Application/
 - Abstractions/
 - Persistence/
 - IApplicationDbContext.cs
 - Authentication/
 - ITokenService.cs
 - ICurrentUser.cs
 - Realtime/
 - IQueueRealtimeNotifier.cs
 - Notifications/
 - INotificationSender.cs
 - Clock/
 - IClock.cs
 - Common/
 - Result.cs
 - Error.cs
 - PagedResult.cs
 - Validation/
 - Behaviors/
 - Features/
 - Auth/
 - Organizations/
 - Branches/
 - Services/
 - Queues/
 - Tickets/
 - Dashboard/

Reports/
Notifications/

QueueFlow.Infrastructure/
Persistence/
ApplicationDbContext.cs
Configurations/
Migrations/
Interceptors/
Authentication/
JwtTokenService.cs
CurrentUser.cs
Realtime/
QueueHub.cs
SignalRQueueRealtimeNotifier.cs
Notifications/
InAppNotificationSender.cs
WhatsAppNotificationSender.cs
Caching/
RedisCacheService.cs
Jobs/
QueueMetricsJob.cs
NotificationDispatchJob.cs
DependencyInjection.cs

QueueFlow.Api/
Controllers/
AuthController.cs
OrganizationsController.cs
BranchesController.cs
ServicesController.cs
QueuesController.cs
TicketsController.cs
DashboardController.cs
ReportsController.cs
Middleware/
ExceptionHandlingMiddleware.cs
TenantResolutionMiddleware.cs
CorrelationIdMiddleware.cs
Filters/
Contracts/
Program.cs
appsettings.json

tests/
QueueFlow.Domain.Tests/
QueueFlow.Application.Tests/

5. Entidades e responsabilidades

Classe	Responsabilidade
Organization	Tenant principal. Nome, slug, documento opcional, timezone, status e plano.
Branch	Unidade física/virtual pertencente a Organization. Endereço, timezone, status.
AppUser	Usuário autenticável. Nome, e-mail, hash/sistema Identity e status.
UserBranch	Relaciona usuário, unidade e papel operacional.
Service	Tipo de atendimento: consulta, corte, retirada, exame etc. Duração média e prefixo.
Queue	Fila operacional de uma unidade/serviço. Estado, estratégia e capacidade.
QueueCounter	Guichê/mesa/sala/posição de atendimento.
QueueTicket	Senha emitida. Estado, prioridade, timestamps, posição lógica e cliente opcional.
TicketEvent	Histórico imutável de transições do ticket.
Notification	Notificação a enviar/enviada, canal, status e tentativas.
Subscription	Plano e situação de cobrança da organização.
AuditLog	Auditoria de ações administrativas e operacionais.

5.1 QueueTicket - campos essenciais

Id : Guid
OrganizationId : Guid
BranchId : Guid
QueueId : Guid
ServiceId : Guid
TicketNumber : string
SequenceNumber : long
Status : TicketStatus
Priority : TicketPriority
CustomerName : string?
CustomerPhone : string?
CustomerPublicToken : string
IssuedAt : DateTimeOffset
CalledAt : DateTimeOffset?
ServiceStartedAt : DateTimeOffset?
CompletedAt : DateTimeOffset?
CancelledAt : DateTimeOffset?
CounterId : Guid?
AttendantUserId : Guid?
RowVersion : byte[] / concurrency token

Nunca usar apenas o número visível da senha como identificador. O cliente deve acessar o ticket por token público aleatório e não enumerável.

5.2 Máquina de estados do ticket

Waiting -> Called -> InService -> Completed

```
|      |      |  
+-----> Cancelled <----+  
|  
+-----> NoShow
```

Called -> Waiting (recall/requeue, conforme regra)

Waiting -> Transferred (opcional na fase 2)

Cada transição deve validar o estado atual. Ex.: um ticket Completed não pode voltar a Waiting sem uma operação administrativa explícita e auditada.

6. Multi-tenancy

Modelo recomendado para o MVP: banco compartilhado e schema compartilhado, com OrganizationId em todas as tabelas de negócio. É econômico e simples de operar. O isolamento deve ser reforçado em várias camadas.

- OrganizationId obtido do usuário autenticado ou contexto público validado.
- Global Query Filters no EF Core para entidades tenant-aware.
- Application services nunca aceitam OrganizationId arbitrário vindo do frontend para operações autenticadas.
- Índices compostos iniciando por OrganizationId nas consultas mais frequentes.
- Testes de integração específicos para impedir vazamento entre tenants.
- Auditoria registra OrganizationId, UserId, IP/correlation ID e ação.

7. Banco de dados e índices

Tabela	Índices importantes
Organizations	UNIQUE Slug
Branches	OrganizationId + IsActive
Services	OrganizationId + BranchId + IsActive
Queues	OrganizationId + BranchId + Status
QueueTickets	OrganizationId + QueueId + Status + Priority + SequenceNumber
QueueTickets	UNIQUE OrganizationId + QueueId + TicketNumber + business date
TicketEvents	TicketId + CreatedAt
Notifications	Status + NextAttemptAt
AuditLogs	OrganizationId + CreatedAt

Para PostgreSQL, usar xmin ou uma coluna de versão conforme estratégia escolhida. Para SQL Server, usar rowversion. A escolha deve ser consistente com o provider final.

8. Casos de uso backend

Use Case	Descrição
CreateOrganization	Cria tenant, administrador inicial e assinatura trial.

CreateBranch	Cria unidade no tenant atual.
CreateService	Cria serviço e configura prefixo/duração.
OpenQueue	Abre fila para atendimento.
IssueTicket	Emite ticket com sequência atômica e token público.
GetPublicTicketStatus	Retorna posição e estimativa sem expor PII.
CallNextTicket	Seleciona atômicamente o próximo ticket elegível.
StartService	Marca início efetivo do atendimento.
CompleteTicket	Finaliza atendimento e atualiza métricas.
CancelTicket	Cancela com motivo.
MarkNoShow	Marca ausência.
RecallTicket	Repete chamada com limite/regra.
GetQueueDashboard	Retorna estado operacional.
GetOperationalReport	Agrega SLA, espera, atendimento e abandono.

9. Algoritmo crítico: chamar o próximo

Este é um dos pontos que exige maior cuidado. Dois atendentes não podem receber a mesma senha. A seleção deve ocorrer em transação com controle de concorrência.

BEGIN TRANSACTION

1. Filtrar:

OrganizationId = current tenant

QueueId = requested queue

Status = Waiting

2. Ordenar:

Priority DESC

IssuedAt ASC

SequenceNumber ASC

3. Bloquear/selecionar o primeiro registro elegível.

4. Alterar Status = Called.

5. Definir CalledAt, CounterId e AttendantUserId.

6. Inserir TicketEvent.

7. Commit.

8. Após commit:

publicar evento SignalR;

enfileirar notificações;

atualizar cache/métricas.

No PostgreSQL, pode-se usar SELECT ... FOR UPDATE SKIP LOCKED quando apropriado. Em EF Core, a implementação pode exigir SQL específico ou uma estratégia equivalente. Criar teste de concorrência com múltiplas chamadas simultâneas.

10. Estimativa de espera

MVP: usar média móvel de duração recente do serviço e quantidade de tickets à frente, considerando número de atendentes ativos. A previsão deve ser apresentada como estimativa, nunca como promessa.

```
estimatedMinutes =  
(ticketsAhead * rollingAverageServiceMinutes)  
/ max(activeAttendants, 1)
```

Fase posterior: segmentar por unidade, serviço, dia da semana e faixa horária; usar percentis e, somente com dados suficientes, modelo preditivo.

11. API REST proposta

Método	Endpoint	Finalidade
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/refresh	Refresh token
GET	/api/v1/branches	Listar unidades
POST	/api/v1/branches	Criar unidade
GET	/api/v1/services	Listar serviços
POST	/api/v1/services	Criar serviço
POST	/api/v1/queues	Criar fila
POST	/api/v1/queues/{id}/open	Abrir fila
POST	/api/v1/queues/{id}/pause	Pausar fila
POST	/api/v1/queues/{id}/close	Fechar fila
POST	/api/v1/public/queues/{publicId}/tickets	Entrar na fila
GET	/api/v1/public/tickets/{token}	Acompanhar ticket
POST	/api/v1/queues/{id}/call-next	Chamar próximo
POST	/api/v1/tickets/{id}/start	Iniciar atendimento
POST	/api/v1/tickets/{id}/complete	Concluir
POST	/api/v1/tickets/{id}/no-show	Marcar ausência
POST	/api/v1/tickets/{id}/cancel	Cancelar
GET	/api/v1/dashboard/queues/{id}	Dashboard
GET	/api/v1/reports/operations	Relatório operacional

11.1 Padrão de resposta

Success:

```
{  
  "data": { ... },  
  "meta": { ... },  
  "correlationId": "..."  
}
```

Error (ProblemDetails):

```
{  
  "type": "...",  
  "title": "Validation error",  
  "status": 400,  
  "detail": "...",  
}
```

```
"errors": { "field": ["message"] },  
"correlationId": "..."  
}
```

12. Autenticação e autorização

- ASP.NET Core Identity ou implementação compatível para usuários administrativos/atendentes.
- Access token JWT curto + refresh token rotacionado e revogável.
- Roles iniciais: Owner, Admin, Manager, Attendant, Viewer.
- Políticas por capacidade, não apenas por role, quando o sistema evoluir.
- Rate limiting em login, emissão pública de ticket e endpoints públicos.
- Cookies HttpOnly são preferíveis quando frontend e backend permitirem arquitetura segura; se JWT no browser, evitar armazenamento inseguro.
- Segredos fora do repositório; usar Secret Manager local e secret store na nuvem.

13. SignalR e tempo real

13.1 Hubs

QueueHub

JoinQueueGroup(queuePublicId)

LeaveQueueGroup(queuePublicId)

DisplayHub (pode ser unificado inicialmente)

JoinBranchDisplay(branchPublicId)

13.2 Eventos servidor -> cliente

ticket.issued

ticket.called

ticket.recalled

ticket.started

ticket.completed

ticket.cancelled

queue.updated

queue.paused

queue.closed

metrics.updated

O servidor é a fonte da verdade. Eventos SignalR apenas notificam mudança; clientes devem poder reconsultar o estado via REST após reconexão.

14. Frontend

apps/

admin-web/

app/

(auth)/

dashboard/
branches/
services/
queues/
users/
reports/
settings/
components/
features/
lib/
hooks/
services/
types/

customer-web/
app/
 q/[queuePublicId]/
 ticket/[publicToken]/
 components/
 lib/

display-web/
app/
 display/[branchPublicId]/
 components/

packages/
 ui/
 api-client/
 shared-types/
 validation/

14.1 Telas do Admin

- Login e recuperação.
- Dashboard geral.
- Organizações/configurações.
- Unidades.
- Serviços.
- Filas.
- Usuários e permissões.
- Relatórios.
- Assinatura/plano.

14.2 Tela do atendente

- Fila selecionada e guichê atual.
- Quantidade aguardando.
- Ticket atual.

- Chamar próximo.
- Rechamar.
- Iniciar atendimento.
- Concluir.
- No-show.
- Transferir (fase 2).

14.3 Jornada do cliente

1. Abrir URL/QR Code da fila.
2. Visualizar estabelecimento, serviço, horário e estimativa.
3. Informar dados mínimos configurados.
4. Confirmar entrada.
5. Receber senha e token público.
6. Acompanhar posição em tempo real.
7. Receber alerta quando estiver próximo.
8. Visualizar chamada e local de atendimento.
9. Após conclusão, opcionalmente avaliar atendimento.

15. QR Code

Cada fila pública deve possuir PublicId não sequencial. O QR Code aponta para a URL do customer-web. O backend não deve confiar apenas no identificador da URL; deve validar status da fila, tenant, horário e regras de capacidade.

`https://app.exemplo.com/q/{queuePublicId}`

16. Notificações

Implementar via abstração INotificationSender e outbox/fila de processamento. O fluxo de atendimento não deve falhar porque WhatsApp ou SMS está indisponível.

Canal	MVP	Fase posterior
In-app/SignalR	Sim	Sim
E-mail	Opcional	Sim
WhatsApp	Não obrigatório	Sim
SMS	Não	Opcional
Push/PWA	Não	Sim

17. Background jobs e Outbox

- NotificationDispatchJob: envia mensagens pendentes e aplica retry exponencial.
- QueueMetricsJob: consolida métricas históricas.
- ExpiredTicketJob: trata tickets expirados conforme regra.
- OutboxProcessor: publica eventos de integração após commit do banco.
- CleanupJob: retenção de tokens, logs e dados temporários.

Hangfire, Quartz.NET ou worker dedicado podem ser usados. No MVP, escolher apenas um mecanismo.

18. Observabilidade

- Structured logging com Serilog.
- Correlation ID em cada request.
- OpenTelemetry para traces e métricas.
- Health checks de API, banco, Redis e jobs.
- Métricas: request latency, error rate, active queues, tickets waiting, calls/min, notification failures.
- Alertas para erro elevado, indisponibilidade e backlog anormal.

19. Segurança e LGPD

- Coletar o mínimo de dados pessoais necessário.
- Telefone e nome do cliente devem ser opcionais conforme configuração do estabelecimento.
- Definir política de retenção e anonimização de tickets históricos.
- Criptografia em trânsito obrigatória.
- Backups criptografados.
- Rate limiting e proteção contra abuso de emissão de tickets.
- Não retornar telefone completo em telas públicas.
- Logs não devem conter tokens, senhas ou PII desnecessária.
- Endpoints administrativos exigem tenant + autorização.
- Auditar alterações de usuários, permissões, configurações e operações administrativas críticas.

20. Testes

Projeto	Cobertura esperada
Domain.Tests	Invariantes, estados e value objects
Application.Tests	Use cases, validação, autorização, regras
IntegrationTests	EF Core, API, auth, tenant isolation, transações
ArchitectureTests	Dependências entre camadas e convenções
Frontend	Componentes críticos e fluxos
E2E	Entrar na fila -> chamar -> iniciar -> concluir

20.1 Cenários obrigatórios

- Dois atendentes chamando simultaneamente não recebem o mesmo ticket.
- Usuário do Tenant A não lê/altera dados do Tenant B.
- Fila pausada/fechada não aceita novo ticket.
- Ticket concluído não pode ser concluído novamente.
- Reconexão SignalR recupera estado via REST.
- Falha de WhatsApp não desfaz chamada de ticket.
- Refresh token reutilizado após rotação é rejeitado.
- Rate limit bloqueia abuso sem impedir fluxo normal.

21. Docker e ambientes

```
docker-compose.yml
api
```

postgres
redis
worker (quando separado)
admin-web
customer-web
display-web

Ambientes: Development, Staging e Production. Migrations devem ser executadas por etapa controlada de deployment, evitando migration automática concorrente em múltiplas instâncias.

22. CI/CD

Pull Request:

restore
build
lint
unit tests
architecture tests
frontend tests

Main:

integration tests
build Docker images
security/dependency scan
push registry
deploy staging
smoke tests
approval
deploy production

23. Roadmap de implementação

Etapa	Entrega
Fase 0 - Fundação	Solution, Git, Docker, padrões, CI, banco, health check.
Fase 1 - Identity/Tenant	Organization, Branch, User, roles, JWT/refresh, tenant context.
Fase 2 - Catálogo	Services, Counters, configurações e CRUDs.
Fase 3 - Queue Core	Queue, Ticket, estados, sequência, emissão pública.
Fase 4 - Atendimento	CallNext, Start, Complete, NoShow, auditoria.
Fase 5 - Realtime	SignalR, customer status, attendant updates, display.
Fase 6 - Frontends	Admin, attendant, customer e display.
Fase 7 - Métricas	Dashboard, relatórios e estimativa.
Fase 8 - Notificações	Outbox, jobs, WhatsApp/e-mail conforme integração.
Fase 9 - SaaS	Planos, limites, trial e billing.
Fase 10 - Hardening	LGPD, rate limit, observabilidade, carga, backup.
Fase 11 - Produção	Staging, domínio, TLS, deploy, smoke tests e monitoramento.

24. Ordem exata para um agente de IA construir

10. Criar repositório e solution .NET; criar projetos Domain, Application, Infrastructure, Api e testes.
11. Configurar referências entre projetos respeitando Clean Architecture.
12. Adicionar packages mínimos e centralizar versões quando possível.
13. Criar BaseEntity, Result/Error e abstrações fundamentais.
14. Criar Organization, Branch, AppUser/UserBranch e configurações EF.
15. Configurar PostgreSQL, DbContext e primeira migration.
16. Implementar autenticação, refresh token, CurrentUser e autorização.
17. Implementar TenantResolution e filtros de tenant; criar testes de isolamento.
18. Criar Service e QueueCounter.
19. Criar Queue, enums e regras de abertura/pausa/fechamento.
20. Criar QueueTicket, TicketEvent e máquina de estados.
21. Implementar sequência de tickets de modo atômico.
22. Implementar IssueTicket e endpoint público.
23. Implementar CallNextTicket com concorrência e teste simultâneo.
24. Implementar StartService, CompleteTicket, CancelTicket, NoShow e Recall.
25. Adicionar SignalR e eventos de atualização.
26. Criar customer-web e jornada QR -> ticket -> acompanhamento.
27. Criar attendant UI e operações em tempo real.
28. Criar display-web.
29. Criar admin-web com unidades, serviços, filas, usuários e configurações.
30. Adicionar estimativa de espera e dashboard.
31. Adicionar Notification abstraction, outbox e background jobs.
32. Adicionar logs, tracing, health checks e métricas.
33. Adicionar rate limiting, políticas de segurança e retenção LGPD.
34. Adicionar Docker Compose e pipeline CI/CD.
35. Executar testes unitários, integração, concorrência, tenant isolation e E2E.
36. Subir staging, executar smoke/load tests e somente depois liberar produção.

25. Regras de Clean Code para geração automática

- Uma classe deve possuir responsabilidade claramente identificável.
- Evitar GenericRepository se ele apenas duplicar DbSet sem regra útil.
- Nomes expressam intenção; evitar Manager, Helper e Utils genéricos.
- Métodos curtos, mas não fragmentar lógica apenas para reduzir linhas.
- Não colocar regra de negócio em Controller.
- Não retornar entidades EF diretamente pela API; usar contracts/DTOs.
- Não criar interface para toda classe automaticamente; abstrair fronteiras e comportamentos substituíveis.
- Validação de formato na borda; invariantes de negócio no domínio/application.
- CancellationToken em operações assíncronas de I/O.
- DateTimeOffset e IClock para tempo testável.
- Não usar .Result/.Wait() em código assíncrono.
- Nullable reference types habilitado e warnings tratados corretamente.
- Não silenciar warnings com ! sem justificativa.
- Paginação em endpoints de listas potencialmente grandes.

26. Definition of Done

- Código compila sem warnings relevantes.
- Testes da feature passam.
- Migration revisada quando houver alteração de schema.
- Autorização e isolamento de tenant testados.
- Erros retornam ProblemDetails consistente.
- Logs possuem correlation ID e não vazam PII.
- OpenAPI atualizado.
- Frontend trata loading, erro, vazio e reconexão.
- Feature testada em Docker/local.
- Documentação e README atualizados.

27. MVP comercial

Para colocar o produto em clientes piloto, o MVP não precisa conter IA, microserviços ou aplicativo nativo. O conjunto mínimo vendável é:

- Cadastro da empresa/unidade.
- Serviços e filas.
- Atendentes/guichês.
- QR Code.
- Entrada do cliente na fila.
- Senha digital e posição.
- Chamar/iniciar/concluir/no-show.
- Atualização em tempo real.
- Painel público.
- Dashboard operacional básico.
- Auditoria e segurança mínima de produção.

28. Evoluções após validação

- WhatsApp oficial para aviso de proximidade e chamada.
- Agendamento + fila híbrida.
- Check-in por geolocalização, quando houver justificativa.
- Kiosk/totem com impressão opcional.
- Pesquisa NPS/CSAT.
- Roteamento por skill do atendente.
- Filas prioritárias configuráveis.
- Integração com ERPs/CRMs.
- Webhooks e API pública.
- White-label.
- Billing recorrente.
- Previsão de demanda e escala de atendentes por IA.

29. Prompt mestre para automação

O texto abaixo pode ser fornecido a um agente de desenvolvimento. Recomenda-se executar por fases e exigir build/testes após cada fase, em vez de pedir a geração de todo o sistema em uma única resposta.

Você é o engenheiro responsável por construir o projeto QueueFlow, uma plataforma SaaS multi-tenant de filas digitais. Use este blueprint como especificação normativa.

Regras:

1. Use ASP.NET Core Web API, C#, EF Core, PostgreSQL, React/Next.js, TypeScript e SignalR.
2. Aplique Clean Architecture pragmática, SOLID e Clean Code.
3. Não implemente microserviços no MVP.
4. Garanta isolamento de OrganizationId em todas as operações de negócio.
5. Controllers devem ser finos e regras devem permanecer em Domain/Application.
6. Use DTOs/contracts; não exponha entidades EF.
7. Habilite nullable reference types e não suprima warnings sem causa documentada.
8. Toda feature deve incluir validação, tratamento de erros e testes.
9. CallNextTicket deve ser transacional e seguro contra concorrência.
10. SignalR não substitui REST como fonte da verdade.
11. Toda mudança de estado de ticket deve produzir TicketEvent.
12. Não permita que falhas de notificações revertam o atendimento já confirmado.
13. Use migrations controladas, Docker e CI.
14. Após cada fase: build, execute testes, corrija erros e somente então avance.
15. Não invente requisitos conflitantes com o blueprint. Se houver ambiguidade técnica, escolha a opção mais simples que preserve segurança, consistência e extensibilidade.

Ordem:

Fase 0 Fundação

Fase 1 Identity/Tenant

Fase 2 Catálogo

Fase 3 Queue Core

Fase 4 Atendimento

Fase 5 Realtime

Fase 6 Frontends

Fase 7 Métricas

Fase 8 Notificações

Fase 9 SaaS/Billing

Fase 10 Hardening

Fase 11 Produção

Para cada fase:

- mostre arquivos criados/alterados;
- entregue classes completas, não fragmentos desconectados;
- explique decisões arquiteturais relevantes;
- execute/valide build e testes quando o ambiente permitir;
- liste comandos de migration e execução;
- não avance deixando erro de compilação conhecido;
- preserve compatibilidade com o código existente.

30. Critérios de aceite fim a fim

37. Uma organização nova consegue criar unidade, serviço, fila e atendente.
38. Um QR Code válido abre a fila correta.
39. O cliente entra e recebe uma senha única.
40. Dois clientes simultâneos não recebem a mesma sequência.
41. O cliente acompanha posição e estimativa.
42. O atendente chama o próximo sem risco de duplicidade.
43. O painel público recebe a chamada em tempo real.
44. O cliente recebe atualização em tempo real.
45. O atendimento pode ser iniciado e concluído.
46. O histórico registra todas as transições.
47. O dashboard apresenta espera, atendimento, abandono e volume.
48. Tenant A não acessa qualquer dado privado de Tenant B.
49. Falha em notificação externa não corrompe o estado do ticket.
50. Sistema reiniciado recupera estado persistido e clientes podem resincronizar.
51. Aplicação passa pelos testes e health checks antes do deploy.

31. Checklist para produção

- Domínio e TLS configurados.
- Banco gerenciado e backups automáticos testados.
- Redis com política de disponibilidade adequada.
- Secrets fora de arquivos versionados.
- CORS restrito.
- Rate limits definidos.
- Logs e traces centralizados.
- Alertas e health checks ativos.
- Política de retenção/LGPD definida.
- Termos de uso e política de privacidade revisados juridicamente.
- Plano de rollback documentado.
- Teste de restauração de backup realizado.
- Teste de carga executado no fluxo IssueTicket/CallNext.
- Smoke test automatizado após deploy.

32. Conclusão

O projeto deve começar como monólito modular, multi-tenant e orientado a eventos internos, priorizando consistência da fila, isolamento entre empresas e experiência em tempo real. O maior risco técnico do domínio não é o CRUD, mas concorrência, transições de estado, sincronização em tempo real, isolamento de tenant e confiabilidade das integrações. O blueprint foi estruturado para que um agente automatizado possa construir o sistema incrementalmente sem perder essas propriedades.